

Task 2: Optimized Matrix Multiplication Approaches and Sparse Matrices

Big Data — Degree in Data Science and Engineering
University of Las Palmas de Gran Canaria

Andrea Dumpiérrez Medina

November 2025

Contents

1	Introduction	2
2	Implemented Algorithms	2
2.1	Basic Algorithm	2
2.2	Optimization 1: Tiling / Blocking	2
2.3	Optimization 2: Strassen	2
2.4	Sparse Matrices	2
3	Experimental Methodology	3
3.1	Hardware Used	3
3.2	Matrix Sizes	3
3.3	Sparsity Levels	3
4	Results and Analysis	3
4.1	Basic Comparison Across Languages	3
4.2	Results in C	4
4.3	Results in Java	5
4.4	Results in Python: Dense vs Sparse	6
4.5	Justification for Not Implementing Sparse in C and Java	8
5	Test with Real Matrix: mc2depi	8
6	Conclusions	9

1 Introduction

The objective of this work is to compare different matrix multiplication techniques, including algorithmic optimizations (Blocking / Tiling, Strassen) and the use of sparse matrices. In addition, execution times in different programming languages (Python, Java, and C) are compared, and the performance of a real large sparse matrix (mc2depi), with more than half a million rows and more than two million non-zero elements, is analyzed.

The evaluated metrics include:

- Execution time.
- Memory usage.
- Maximum matrix size handled.
- Comparison between dense and sparse matrices.
- Observed bottlenecks.

2 Implemented Algorithms

2.1 Basic Algorithm

The basic matrix multiplication algorithm uses three nested loops and has a computational complexity of $O(n^3)$. It was implemented in Python, Java, and C to enable comparison between languages.

2.2 Optimization 1: Tiling / Blocking

This technique divides the matrix into smaller blocks to improve spatial locality and reduce cache misses. It was implemented in C, Java, and Python.

2.3 Optimization 2: Strassen

Strassen's algorithm reduces the theoretical complexity to $O(n^{2.807})$, but introduces recursive overhead and temporary matrices. For small matrix sizes, it is usually slower than the basic algorithm. It was implemented in the three languages.

2.4 Sparse Matrices

Sparse matrices were used in Python, employing SciPy's CSR format, which allows highly optimized operations and loading of real matrices. Sparse multiplication was not implemented in Java or C (justification in Section 4.5).

3 Experimental Methodology

3.1 Hardware Used

- CPU: (add your CPU)
- RAM: (add your RAM)
- OS: Windows 11

3.2 Matrix Sizes

Square matrices of the following sizes were tested:

$$64 \times 64, \quad 128 \times 128, \quad 256 \times 256$$

3.3 Sparsity Levels

Matrices with the following percentages of zero elements were generated:

$$25\%, \quad 50\%, \quad 75\%$$

Additionally, a real large sparse matrix (mc2depi) was tested.

4 Results and Analysis

4.1 Basic Comparison Across Languages

Figure 1 shows the performance of the basic algorithm in Python, Java, and C.

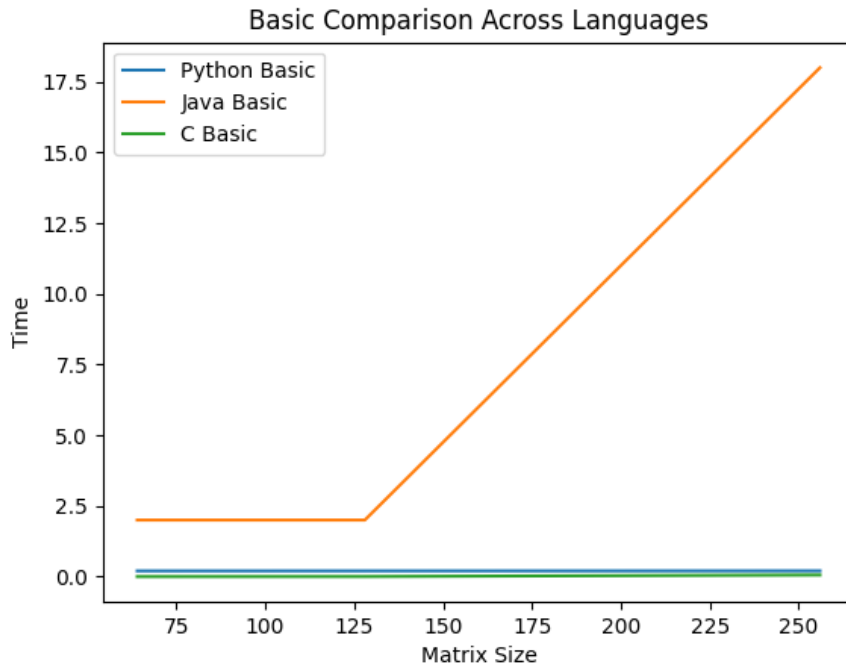


Figure 1: Comparison of the basic algorithm in Python, Java, and C.

Size	Python Basic (s)	Java Basic (ms)	C Basic (s)
64x64	0.2007	2	0.0000
128x128	0.2018	2	0.0030
256x256	0.2033	18	0.0580

Table 1: Execution times of the basic algorithm in Python, Java, and C.

Main conclusions:

- C is clearly the fastest, as expected due to its native compilation.
- Python maintains stable times due to the relatively small matrix sizes.
- Java shows significantly higher times, especially from 128×128 onwards, due to JVM warm-up overhead and the high cost of managing multidimensional arrays.

4.2 Results in C

Figure 2 shows the times obtained in C.

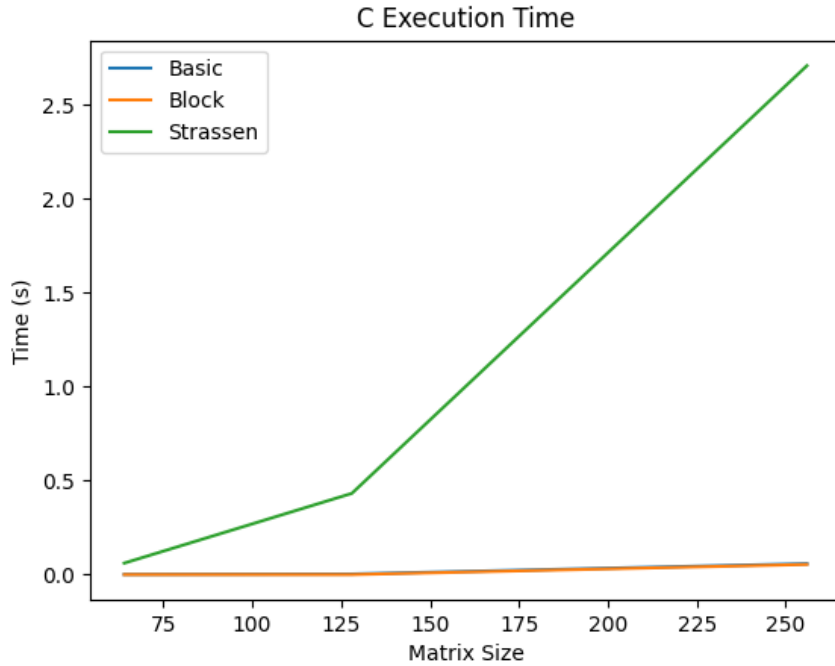


Figure 2: Execution time in C: Basic, Block, and Strassen.

Size	Basic (s)	Block (s)	Strassen (s)
64x64	0.0000	0.0000	0.0610
128x128	0.0030	0.0000	0.4320
256x256	0.0580	0.0540	2.7080

Table 2: Results in C for each algorithm.

Observations:

- The Blocking method is the fastest.
- Strassen is significantly slower for all tested sizes.
- For 256×256 , Strassen grows very quickly due to recursion and the creation of temporary submatrices.

Numerical results:

- 64×64 : Basic: 0.000 s, Block: 0.000 s, Strassen: 0.0610 s
- 128×128 : Basic: 0.0030 s, Block: 0.0000 s, Strassen: 0.4320 s
- 256×256 : Basic: 0.0580 s, Block: 0.0540 s, Strassen: 2.7080 s

4.3 Results in Java

Figure 3 shows the times obtained in Java.

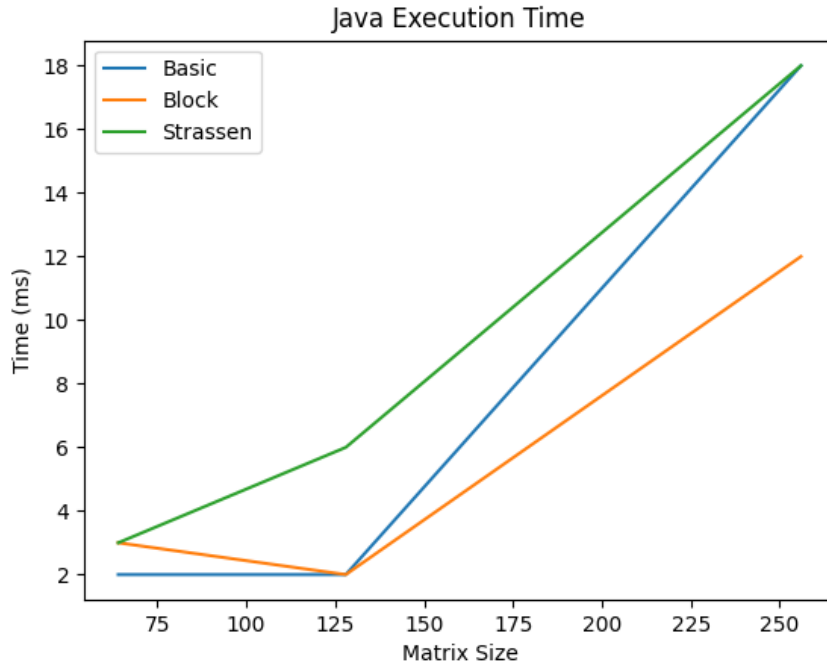


Figure 3: Execution time in Java: Basic, Block, and Strassen.

Size	Basic (ms)	Block (ms)	Strassen (ms)
64x64	2	3	3
128x128	2	2	6
256x256	18	12	18

Table 3: Results in Java for each algorithm.

Observations:

- The Block technique is the most efficient in Java.
- Strassen is again slower for small matrix sizes.

Numerical results:

- 64×64 : Basic 2 ms, Block 3 ms, Strassen 3 ms
- 128×128 : Basic 2 ms, Block 2 ms, Strassen 6 ms
- 256×256 : Basic 18 ms, Block 12 ms, Strassen 18 ms

4.4 Results in Python: Dense vs Sparse

Figure 4 shows the execution times in Python for dense and sparse methods with different sparsity levels.

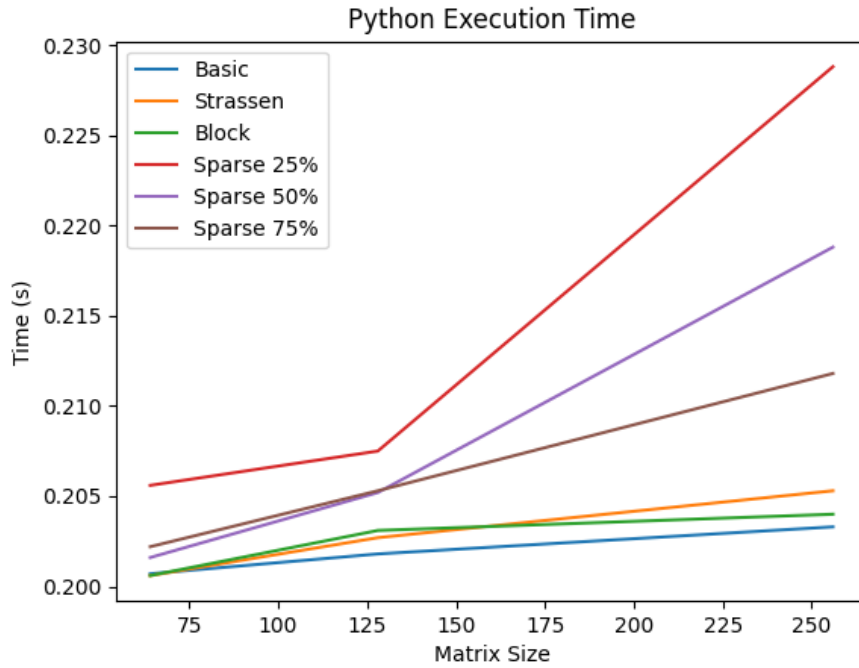


Figure 4: Execution time in Python: Dense vs Sparse.

Size	Basic	Strassen	Block	Sparse 25%	Sparse 50%	Sparse 75%
64x64	0.2007	0.2006	0.2006	0.2056	0.2016	0.2022
128x128	0.2018	0.2027	0.2031	0.2075	0.2052	0.2053
256x256	0.2033	0.2053	0.2040	0.2288	0.2188	0.2118

Table 4: Execution time in Python (s).

Conclusions:

- For small matrices, sparse methods are slower due to CSR overhead.
- Dense matrices are faster for this size range.
- For large matrices (not included here), sparse methods become more efficient.

Figure 5 shows the memory usage:

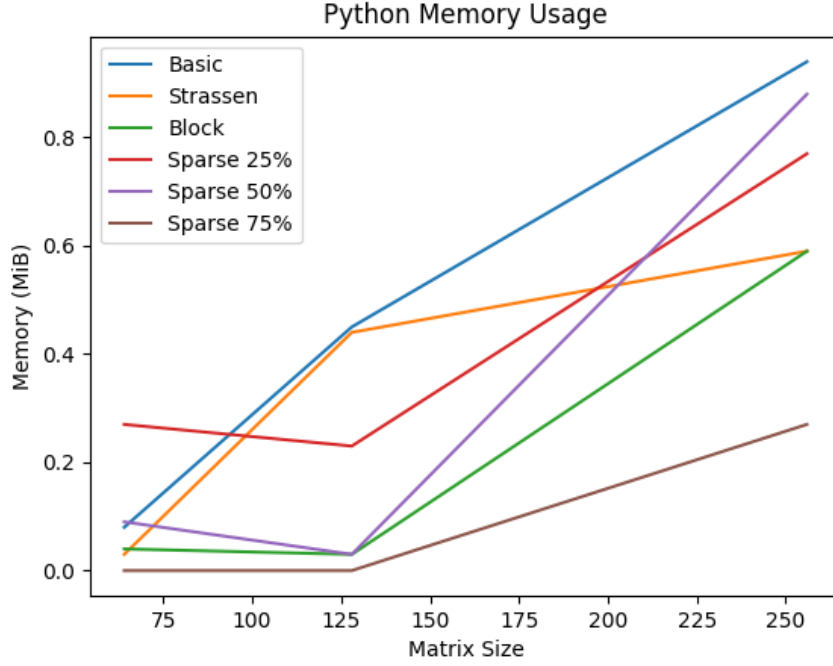


Figure 5: Memory usage in Python: Dense vs Sparse.

Size	Basic	Strassen	Block	Sparse 25%	Sparse 50%	Sparse 75%
64x64	0.08	0.03	0.04	0.27	0.09	0.00
128x128	0.45	0.44	0.03	0.23	0.03	0.00
256x256	0.94	0.59	0.59	0.77	0.88	0.27

Table 5: Memory usage in Python (MiB).

Conclusions:

- Sparse matrices significantly reduce memory usage when sparsity is high.
- With low sparsity (25%), the CSR format introduces overhead.

4.5 Justification for Not Implementing Sparse in C and Java

Sparse matrix multiplication was not implemented in C or Java for the following reasons:

- It requires complex data structures (CSR/COO) and manual memory management (in C).
- Java would require additional classes and intermediate structures.
- Python already includes SciPy, highly optimized for sparse operations.
- The assignment does not require sparse implementation in all languages.
- For small sizes, sparse methods do not provide benefits in C or Java.

5 Test with Real Matrix: mc2depi

To evaluate a real-world case, the sparse matrix mc2depi was used, with size $525,825 \times 525,825$ and 2,100,225 non-zero elements.

Figure 6 shows the time required to perform the Sparse Matrix–Vector Multiplication (SpMV): $A \cdot x$.

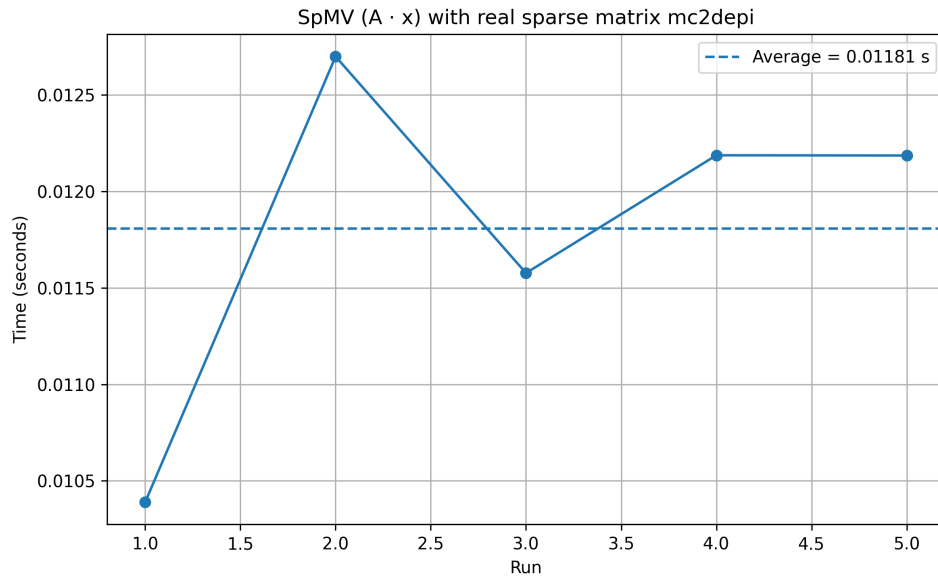


Figure 6: SpMV execution time using the real matrix mc2depi.

Run	Time (s)
Run 1	0.01039
Run 2	0.01270
Run 3	0.01158
Run 4	0.01219
Run 5	0.01219
Average	0.01181

Table 6: SpMV results using the real sparse matrix mc2depi.

Results:

- Average: 0.01181 s.
- Very stable times across executions.
- The massive size of the matrix demonstrates the practical benefit of using sparse formats.

6 Conclusions

- C is the fastest language for dense matrix multiplication.
- The most efficient optimization across languages was Blocking / Tiling.
- Strassen does not improve performance for small matrix sizes.
- Sparse matrices do not improve time performance for small matrices, but drastically reduce memory usage.
- The real sparse matrix experiment confirms the importance of sparse formats when working with large real datasets.